CITY BRAIN / TECHNICAL STUDY

Building the City Brain twin layer

Cities: Skylines II

Modding options, working interfaces, and a first experiment

The useful part of this idea is the feedback loop: something happens in the city, City Brain notices it, and a response changes what happens next. The game already supplies much of the visible world. The engineering work is in making that loop observable, controllable, and repeatable.

This revised report follows that work down to the code. It explains the methods behind the original wiki link, shows the equivalent starting point in Cities: Skylines II, and gives commands for an existing community bridge. Fire, flooding, traffic, and everyday routines are treated as specific integration problems.

Platform: Cities: Skylines II on PC
Revised: 5 October 2026
Research basis: developer documentation and public mod source code

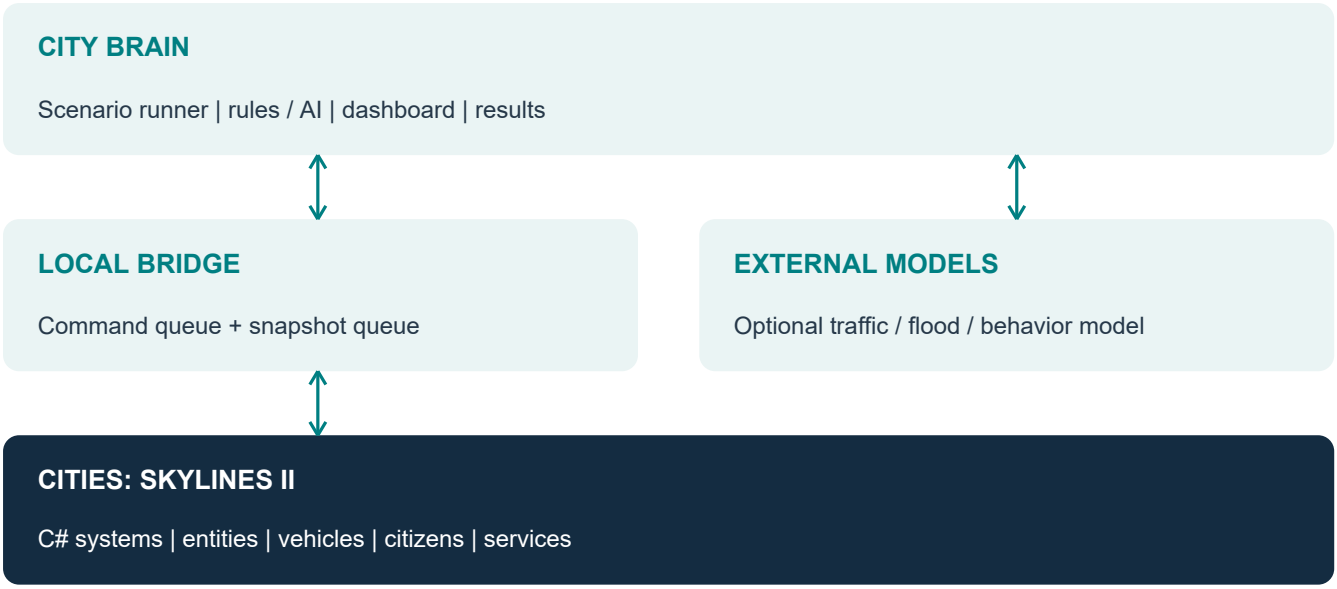
The examples are implementation guidance, not a tested game installation. The C# snippets require the relevant game assemblies. HTTP commands require the named community bridge; they are not built-in game commands.

Read pages 3-4 for the original wiki API, pages 5-11 for the technical walkthrough, and pages 12-17 for the experiment design. The 36-scenario catalog is retained on pages 18-20. Sources follow on pages 21-24.

What I would build first

I would begin with one busy junction, a saved starting city, and a dashboard that reads the current state. Once that works, I would let the dashboard change one junction control and record the result. That is a small enough experiment to finish, but it still proves the essential twin-layer architecture.

Starting with several hazards at once makes it difficult to know why a result changed. A different work schedule, a population mod, and a road change can each alter traffic before an incident even begins. Establish the baseline first; introduce one cause at a time.



The C# bridge belongs inside the game. Scenario timing, storage, and decision logic belong in the external application. Specialist models can be added later when the game's approximation is too coarse. A flood model, for example, could calculate a hazard footprint while the game illustrates its effect on access to a hospital.

CS2MCP is worth evaluating before writing a bridge from scratch. It already exposes useful observations and a number of city controls. Its source also makes the missing work visible: there is no documented universal command for creating any disaster, controlling every citizen, or replaying a whole city deterministically. [17]

Reading the API in your link

The supplied wiki describes the original Cities: Skylines and its **ICities** API. Most entries are C# methods or callbacks, not console commands. You implement a callback in a mod; the game calls it at the relevant point in its lifecycle.

The original site remained inaccessible during research. A published copy helped locate the relevant sections, and the methods below were checked against mod authors' source. This is a legacy reference, not a claim that the same methods exist in CS II. [1][28][29][30][31]

Method / API	What it is useful for
IUserMod.Name / Description	Identify a CS I mod in the game's content interface. [31]
OnLevelLoaded(LoadMode mode)	Start work after a level loads; inspect mode before assuming it is a playable city. [31]
OnBeforeSimulationTick()	Run short work before a simulation tick. TM:PE uses this hook for geometry and routing work. [29]
OnBeforeSimulationFrame()	Hook frame-level simulation work. Keep it short. [29]
OnAfterSimulationTick()	Collect or process state after the tick. [29]
OnSaveData() / OnLoadData()	Persist and restore mod-specific state alongside a save. [30]
SaveData(string id, byte[] data) LoadData(string id)	Write/read a named byte payload through ISerializableData. Use a unique key and a versioned format. [30]

For this project, the important ideas are lifecycle, scheduled work, and saved state. They carry over to CS II even though the types do not. The next page shows the older pattern; the pages after it switch to the edition you own.

Do not paste ICities imports into a CS II project. A callback that observes an event also does not create that event; observation and intervention are separate tasks.

A small example of the original API

This CS I example declares a mod and logs when a normal city is ready. It uses the legacy method names discussed on the preceding page. The code is written for this report; it is not copied from the wiki. [31]

C# / Cities: Skylines I only / illustrative, not compiled here

```
using ICities;
using UnityEngine;

public sealed class TwinNotesMod : IUserMod
{
    public string Name { get { return "Twin Notes"; } }
    public string Description
    {
        get { return "A minimal city-load observation."; }
    }
}

public sealed class TwinNotesLoading : LoadingExtensionBase
{
    public override void OnLevelLoaded(LoadMode mode)
    {
        base.OnLevelLoaded(mode);
        if (mode != LoadMode.NewGame && mode != LoadMode.LoadGame)
            return;

        Debug.Log("Twin Notes: city loaded; start a new run.");
    }
}
```

A telemetry extension could then derive from **ThreadingExtensionBase** and override **OnAfterSimulationTick()**. It should copy a small observation to a queue, not wait for an HTTP request or write a large file on every tick. The save-data extension is where a scenario ID or configuration version could be retained. [29][30]

These hooks explain how an in-game mod becomes a bridge. They do not themselves expose a web server, route an ambulance, or start a fire. Those behaviors must be implemented behind the hooks.

For your actual build, use the CS II example next. This page is included so the original reference is useful without leading you into the wrong modding toolchain.

The starting point in Cities: Skylines II

In CS II, begin with the current official mod template. It supplies assembly references and the post-processing needed by the Unity tooling. A mod registers systems in the game's update loop through **IMod.OnLoad**. [2]

C# / CS II / Mod.cs / pair with BuildingCountSystem on page 6

```
using Game;
using Game.Modding;

namespace CityBrain
{
    public sealed class Mod : IMod
    {
        public void OnLoad(UpdateSystem updates)
        {
            updates.UpdateAt<BuildingCountSystem>(
                SystemUpdatePhase.UIUpdate);
        }

        public void OnDispose()
        {
            // Stop application-owned listeners/workers here.
        }
    }
}
```

The registration method and update phase are used by the inspected CS2MCP implementation. UIUpdate is useful for a bridge that must answer while the game is paused. A paused game can still answer a request, but simulation-derived values may not have advanced. [18][36]

Why the phase matters

Rendering, simulation jobs, tool application, and network requests do not all run at the same point. A reader must respect the dependencies of the data it reads. A writer must also leave the game's related systems consistent. For road edits, prefer the game's tool/upgrade pathway rather than changing one field and assuming pathfinding will repair everything.

This is a minimal registration example, not a complete network mod. It requires the generated project, the game assemblies, and the system class on the next page. No game-side compilation was performed for this report.

Reading entities: a building counter

An entity is an object identity; components hold its data. A query selects the identities carrying the components you need. This small system counts placed buildings and excludes temporary construction previews and deleted entities. It follows the component selection used in the inspected building reader. [34]

C# / CS II / BuildingCountSystem.cs / illustrative, not compiled here

```
using Game;
using Unity.Entities;
using UnityEngine.Scripting;

namespace CityBrain
{
    public partial class BuildingCountSystem : GameSystemBase
    {
        private EntityQuery buildings;
        public int LastBuildingCount { get; private set; }

        [Preserve]
        protected override void OnCreate()
        {
            base.OnCreate();
            buildings = GetEntityQuery(new EntityQueryDesc
            {
                All = new[] {
                    ComponentType.ReadOnly<Game.Buildings.Building>(),
                    ComponentType.ReadOnly<Game.Objects.Transform>(),
                    ComponentType.ReadOnly<Game.Prefabs.PrefabRef>()
                },
                None = new[] {
                    ComponentType.ReadOnly<Game.Tools.Temp>(),
                    ComponentType.ReadOnly<Game.Common.Deleted>()
                }
            });
        }

        [Preserve]
        protected override void OnUpdate()
        {
            LastBuildingCount = buildings.CalculateEntityCount();
        }
    }
}
```

This counts matching entities, including eligible sub-buildings, not households or occupied addresses. It is deliberately a small reader. A production exporter should sample at a chosen cadence and publish immutable results; background network code should not directly inspect EntityManager.

Trying an existing bridge

The following commands target **LancerComet/cities-skylines-2-mcp**, a community project. They are not commands from the Paradox wiki. The reviewed revision is pinned below so the examples can be compared with the same source later. [17][27]

PowerShell / run later in a separate development folder

```
git clone https://github.com/LancerComet/cities-skylines-2-mcp.git
Set-Location cities-skylines-2-mcp
git checkout f0894c1831153742d2a586b830418177839a2428

# Replace this example path with your installed game directory.
$gamePath = 'D:\SteamLibrary\steamapps\common\Cities Skylines II'
dotnet build .\CS2MCP.Bridge\CS2MCP.Bridge.csproj "-p:GamePath=$gamePath"
```

The project's build step deploys the bridge into the local CS II Mods directory. Start or restart the game, load a small test city, then check the bridge. The repository documents Windows and Steam; another storefront needs its path and loading behavior checked. [17][27]

PowerShell / requires the bridge to be loaded in the game

```
$bridge = 'http://127.0.0.1:8642'
Invoke-RestMethod "$bridge/ping"
Invoke-RestMethod "$bridge/state"
```

The README requests .NET SDK 8+ for building, while the project targets **net472**. Those describe different things: the build tool and the target runtime. Game assemblies remain local references. [27]

MCP is optional

The C# bridge can be called directly over HTTP. If an MCP client also needs access, the separate Node server can be built with the commands below. This extra process is not needed for the PowerShell or Python examples in this report.

Optional / Node.js 18+ according to the project README [17]

```
Set-Location .\mcp-server
npm.cmd install
npm.cmd run build
```

The report update did not run these installation commands or alter your game. The repository still contains only the PDF.

Reading the city from PowerShell

Keep the first connection simple. Ask whether a playable city is loaded, then read an aggregate and a small set of objects. These routes are present in the inspected bridge dispatcher and handlers. [32][34]

PowerShell / community HTTP API

```
$bridge = 'http://127.0.0.1:8642'
$state = Invoke-RestMethod "$bridge/state"
if (-not $state.cityLoaded -or $state.isLoading) {
    throw 'Load a playable city before continuing.'
}

Invoke-RestMethod "$bridge/city/overview"
Invoke-RestMethod "$bridge/city/services"
Invoke-RestMethod "$bridge/city/buildings?limit=20"

$traffic = Invoke-RestMethod "$bridge/city/traffic?limit=10"
$traffic.cityTrafficFlowPercent
```

Route	What to inspect
/state	cityLoaded, isLoading and simulation.frameIndex; use these to identify progress or a reload.
/city/overview	Population, average health/happiness and city finance values.
/city/buildings?limit=20	Entity index/version, prefab name and game-world position.
/city/traffic?limit=10	Game flow diagnostics and selected problem roads; not calibrated traffic counts.
/city/notifications?limit=20	Warning icons and their targets; not a complete incident lifecycle log.

A useful first check is to compare one returned building with the visible city. Confirm its position and type, pause the game, and query again. Then reload the save and repeat. This catches wrong-world references early, before they are embedded in an experiment.

Read results as game data. The traffic handler explicitly reproduces game flow/volume diagnostics; a number labeled "volume" must not be reported as vehicles per hour without a separate conversion and validation. [19]

Controlling time and saving a baseline

Time control is more useful for an experiment than a long list of construction actions. It lets a runner start from a known city, advance a bounded amount, and collect observations. The commands below are actual routes and parameters from the inspected handlers. [32][33]

PowerShell / community HTTP API / these calls change game state

```
$bridge = 'http://127.0.0.1:8642'

# Pause, then request a named baseline save.
Invoke-RestMethod "$bridge/sim/control?paused=true"
Invoke-RestMethod "$bridge/game/save?name=CityBrain-baseline"

# Confirm the save has finished in the game before proceeding.
# Start a quarter of a displayed game hour at normal speed.
$run = Invoke-RestMethod "$bridge/sim/run?hours=0.25&speed=1"
$run.startFrame
$run.targetFrame

# Inspect progress; this call does not wait for completion.
Invoke-RestMethod "$bridge/state"

# Cancel the scheduled run and pause.
Invoke-RestMethod "$bridge/sim/run?cancel=true"
```

The timed-run handler returns immediately with a target frame. Saving is asynchronous too: **saving=true** means the operation was requested, not that a usable save already exists. Check completion before treating that file as your baseline. [33]

A clock assumption worth checking

The inspected handler converts hours to frames using **262144 / 24**. That is its implementation assumption. A mod that changes day length or calendar behavior can make "0.25 hours" differ from the duration your scenario expects. Record frame indices as well as displayed time and calibrate the relationship before enabling schedule mods. [33]

This bridge uses query-string requests for changes, including GET calls. Keep them inside the local runner; do not turn them into dashboard links that a previewer or prefetcher could follow. A production wrapper should expose explicit write operations.

A short Python experiment runner

This script advances a quarter-hour, polls until the returned frame target is reached, and writes observations to JSON Lines. It uses only the Python standard library. Save it as **collect_run.py** in the later implementation project and run **python collect_run.py**. It is an original example using the verified routes. [32][33]

Python / illustrative runner; game execution not tested

```
import json, time, uuid
from urllib.request import urlopen

BASE = "http://127.0.0.1:8642"
def call(path):
    with urlopen(BASE + path, timeout=10) as response:
        return json.load(response)

state = call("/state")
if not state["cityLoaded"] or state["isLoading"]:
    raise RuntimeError("Load a city first")

run_id = uuid.uuid4().hex
run = call("/sim/run?hours=0.25&speed=1")
last_frame = run["startFrame"]
deadline = time.monotonic() + 600
try:
    with open(run_id + ".jsonl", "x", encoding="utf-8") as out:
        while time.monotonic() < deadline:
            before = call("/state")
            if not before["cityLoaded"] or before["isLoading"]:
                raise RuntimeError("City unloaded during run")
            frame = before["simulation"]["frameIndex"]
            if frame < last_frame:
                raise RuntimeError("Clock reset; discard this run")
            traffic = call("/city/traffic?limit=10")
            after = call("/state")
            row = dict(run_id=run_id, wall_time=time.time(),
                       before=before, traffic=traffic, after=after)
            out.write(json.dumps(row) + "\n")
            last_frame = frame
            if frame >= run["targetFrame"]:
                break
            time.sleep(1)
        else:
            raise TimeoutError("Run did not reach its target")
finally:
    call("/sim/run?cancel=true")
```

The before/after state records bracket each traffic read; they do not make it atomic. A disconnect can also prevent the final pause request. The next page explains the additions needed for a reliable service.

What the small examples leave out

A working HTTP request is only the first milestone. The harder failures appear when the player reloads a city, the simulation falls behind, or a command times out after it has already changed the world.

Identity and lifecycle

An entity needs both its index and version. Pair that with your own run/world identifier, and rebuild mappings after every load. An index by itself can be reused. The Python example notices some clock resets; it is not a complete detector for every possible reload. A production bridge should expose an explicit world generation or session token.

Consistent observations

Sequential HTTP reads are taken at different updates. For a control decision that combines roads, vehicles, and service state, add a snapshot operation which captures them under one well-defined update boundary. If a game API returns a JobHandle for an array, observe its dependency contract before reading that array. The bridge's demand reader demonstrates an explicit completion step. [32]

Bounded work and acknowledged actions

Use separate input and output queues. Process only a bounded amount of command work per update, attach a deadline to each command, and distinguish queued, applied, rejected, and failed. Never retry a timed-out write blindly. First inspect the target; otherwise one request can become two building placements or two incident starts.

The inspected bridge drains a concurrent request queue inside its system update. That is a useful starting pattern, but a busy dashboard can overload it. Throttle polling, limit result sizes, and keep serialization and disk writing away from the critical update wherever practical. [18]

One controller per mechanism

Let the game own movement unless an external model deliberately replaces it. Let one routine mod own schedules. Let one signal controller own a junction. Two systems writing the same state can produce a convincing animation with no coherent experiment underneath.

For a later service, add command IDs, deduplication, read-back checks, authentication appropriate to its exposure, and explicit POST endpoints. The inspected listener binds to loopback; retain that default for a local demo. [35]

Traffic: a complete first intervention

Use a junction with at least three connected roads. The inspected bridge validates the entity and applies the change through its tool workflow. Its supported modes are **lights**, **nolights**, **stop**, and **default**. This is junction control, not arbitrary green-time programming. [20]

PowerShell / changes one selected junction / start from a saved test city

```
$bridge = 'http://127.0.0.1:8642'

# Choose a center from your city's returned coordinates.
$graph = Invoke-RestMethod (
    "$bridge/city/road-graph?x=0&z=0&radius=300"
)
$graph | ConvertTo-Json -Depth 8

# Use a NODE identity from that result, not a road-segment ID.
$nodeIndex = [int](Read-Host 'Junction node index')
$nodeVersion = [int](Read-Host 'Junction node version')
$query = "index=$nodeIndex&version=$nodeVersion&mode=lights"
Invoke-RestMethod "$bridge/build/junction?$query"
```

Run baseline A without changing the junction. Reload the same save, re-identify the junction, apply the intervention, and run B for the same duration. Re-query the graph after the edit and confirm the control in the game. A rebuild may change identities or lane layout.

For the first result, use the game's traffic-flow metric and label it accordingly. For a stronger result, instrument completed traversals and journey times. Watch the neighboring junction too: clearing one queue can simply move it downstream.

Traffic adds lane connections and priorities. Traffic Lights Enhancement adds predefined sequences, while its published guide excludes duration control. If emergency-vehicle priority requires exact phase timing, that is a separate controller integration. [10][11]

HTTP 400 indicates an invalid request or unsuitable junction; 404 can mean a stale entity; 409 can mean another build operation is active. The source validates these cases. Verify the state before deciding whether to retry. [20]

Fire: observe first, automate second

Fire is attractive because the causal story is easy to see: ignition, dispatch, congestion, arrival, and control. CS II already has fire services and building/forest-fire behavior. The missing piece for City Brain is a reliable way to trigger and track a specific incident. [5][6]

The reviewed bridge has no documented fire-start endpoint. I would not invent one in the report. Begin by observing a rehearsed incident, then inspect the installed build's event and tool pathways to implement the trigger. Spawning the wrong entity or setting a burning flag alone may leave dispatch, damage, and the event journal out of sync.

The adapter I would write

C# sketch / custom types and methods still need implementation

```
// Proposed City Brain interface; these are NOT game API names.
interface IIncidentAdapter
{
    bool CanStart(string kind, string targetRef);
    string Start(string kind, string targetRef);
    IncidentSnapshot Read(string incidentId);
}

// IncidentSnapshot is a project-defined DTO containing:
// run ID, incident ID, tick, location, affected entities,
// observed status, linked response vehicles, data quality.
```

Before exposing Start, test one incident by hand and document the component changes over its lifecycle. Identify how the game creates the event, discovers affected buildings, requests service, and records completion. Then route a command through that same validated pathway. Add a duplicate-start guard and return a stable incident ID.

For the demonstration, place a station and a residential block on opposite sides of one congested junction. Compare native response against a pretested junction intervention. Keep dispatch native until you can reliably associate an engine with its incident; a passing fire engine is not enough evidence of response.

Measure ignition-to-detection, detection-to-dispatch, and dispatch-to-arrival separately. If any timestamp or association is unavailable, leave that metric missing. Game-simulated damage is not a calibrated real-world casualty estimate.

Flooding: water controls and impact maps

A flood experiment needs two separate decisions: how water enters the scene, and when that water makes an asset unusable. Water Features offers water-source editing and seasonal behavior. Its default seasonal streams are not intended to flood the map without changing the source configuration. [14]

Build a simple low area with one access road to a clinic and one higher alternative. Use a controlled source change and wait for water to settle. Export terrain and water depth for inspection, then decide whether the scenario needs physical flood modeling or only a transparent gameplay approximation.

PowerShell / read-only game query; writes a local snapshot [17]

```
$bridge = 'http://127.0.0.1:8642'
$terrain = Invoke-RestMethod "$bridge/city/terrain?resolution=128"
$terrain | ConvertTo-Json -Depth 20 |
    Set-Content -LiteralPath '.\terrain-water.json' -Encoding UTF8
```

The response is game terrain/water data, not a rainfall model. Preserve the returned grid dimensions, bounds and units. Do not guess a road's water depth by using its array index as a coordinate. Map world X/Z into the grid and test the conversion at several visible points.

Turning water into a scenario

Intersect the depth field with roads and service buildings. Define a passability threshold explicitly, including how bridges are handled. Verify that the game actually stops traversal where your scenario says access is lost. If it does not, enforce the closure through a separate validated traffic adapter and record that intervention.

Carto can export vector assets and terrain/water rasters, and provides a callable export API for other mods. It is useful for analysis in GIS, but a file export is not a live flood-control command. [15][16]

For rainfall and drainage analysis, investigate SWMM or another suitable model. A complete two-dimensional flood surface may need additional coupling. Let that model own the hazard calculation, then transfer impacts to the city. Keep this outside the first hackathon build unless flood physics is the central judging criterion. [24]

People, schedules, and useful mods

Daily routines make incidents meaningful. A road closure at night and the same closure during school arrival should not be treated as the same experiment. CS II supplies citizens and households; Realistic Trips gives more control over the timing of their journeys. [3][12]

Project	Why I would consider it
Realistic Trips [12]	Work shifts, school schedules, shopping, weekends and remote-work shares. Introduce it before collecting the baseline.
Traffic [10]	Lane connections and priorities. Inspect a junction again after a road upgrade, because edits can reset the custom arrangement.
Water Features [14]	Source placement and water-level experiments. Automating its tools still requires an adapter.
RWH [13]	Building-based household and workplace capacity. Expect residents and demand to adjust after changing settings.
Carto [15]	Export the simulated city for spatial analysis and comparison with real geography.
Skyve [26]	Manage playsets and review compatibility information before testing a combination.

I would start with the bridge alone, add the mod needed for the chosen experiment, and repeat the same load-read-write-reset checks. The Realistic Trips and RWH READMEs target game 1.6.*, while the reviewed Traffic Lights Enhancement README cites 1.3.3f1. Neither statement certifies compatibility with your installed build. [11][12][13]

For human behavior beyond schedules, add a small cohort model outside the game: people with limited mobility, delayed warnings, or different evacuation compliance. Those are your model assumptions. An LLM can help author a scenario or explain measured results; it should not be asked to invent each citizen's next action on every simulation update.

An older mod with a similar name may target CS I or the early CS II loading system. Follow the author's distribution link and inspect the exact version you install.

Geography and the data worth keeping

Recreating a whole city is tempting. For the first prototype, a recognizable neighborhood is usually enough: correct road connections, plausible building capacity, the important services, and a terrain shape that fits the scenario.

Use the native editor for a controlled map. Custom asset support allows more recognizable buildings and props, but appearance alone does not add service behavior. MapToSkylines2 is an interesting OSM pipeline; its author still describes the in-game map-building part as work in progress. The older MapImageLayer repository is archived. [9][21][22]

Preserve a coordinate transform

Store the projected coordinate system, map origin, scale, north direction, and height reference. Test it against landmarks. An exported GeoJSON file is not automatically aligned with the real city just because it has geographic coordinates. Carto lets you configure the projection, which is useful once the transform is known. [15]

A small experiment record

JSON / proposed project schema; placeholders must be filled

```
{
  "run_id": "junction_B_03",
  "baseline_save_hash": "<computed before the experiment>",
  "game_build": "<installed build>",
  "bridge_commit": "f0894c1831153742d2a586b830418177839a2428",
  "scenario_version": 1,
  "intervention": "junction lights",
  "clock_basis": "simulation frame index",
  "mods": [],
  "observations_file": "junction_B_03.jsonl"
}
```

Also record the node identity used for the command, when it was requested and confirmed, the intervention parameters, and any samples that were dropped. For incidents, retain the requested start and the actual observed start separately.

Keep synthetic sensor noise separate from the observed game state. That allows you to test how City Brain behaves with delayed or missing information while still knowing what actually happened in the simulated run.

What would count as a good result

The first demonstration should answer one question well. For a traffic intervention, I would want the same starting save, the same demand settings, a verified control change, and a graph showing what happened before and after it. A more cinematic disaster is not automatically a stronger result.

Repeat baseline and intervention runs, ideally in alternating order. Three to five pairs are a useful hackathon target, not a statistical guarantee. Show the individual results as well as their average. A saved city and a seed do not establish deterministic replay across every mod or asynchronous game system.

Measure	How to avoid a misleading result
Trip time	Use matched departures and arrivals; show sample count and unfinished trips.
Response time	Separate detection, dispatch and travel; verify the vehicle-to-incident link.
Access loss	State the graph, road restrictions and affected population assumptions.
Traffic improvement	Check adjacent roads and completed trips, not just one flow percentage.
Runtime cost	Record observation age, request latency, lost samples and effective simulation speed.

How I would spend the hackathon

First get a clean city and a stable reader. Then implement one write and read it back. Use the next block of time for the runner, logging and reset procedure. Spend the final block repeating the experiment and rehearsing the presentation. If an incident trigger is still unreliable, use a labeled manual trigger and demonstrate the live detection and response around it.

Keep utility and economic assumptions tied to the installed build: Economy 2.0 changed service-import behavior. Likewise, the announced First Response content is not evidence of an available incident API. Do not make the minimum prototype depend on it. [7][8][25]

If the work later becomes a transport research project, SUMO and its TraCI interface deserve a closer look. CS II can remain the city context, but coupling two simulators is a new engineering task with its own ownership and synchronization rules. [23]

Scenario ideas: movement and routine

These are directions to explore after the first experiment. N = native game mechanism, M = community mod, C = custom code, X = external model. Effort: L = setup, M = adapter work, H = a new model or deeper integration. Native behavior does not imply an automated trigger.

Scenario	Route / effort	Proposed trigger and useful outcome
1. Morning commute	N + M / L	Tune work/school schedules; compare peak queues and sampled trip times. [3][12]
2. Staggered shifts	M / L	Spread start times; compare congestion and completed trips.
3. School arrival surge	M + C / M	Change attendance/start windows; measure access around schools.
4. Remote-work policy	M / L	Vary remote-work share; measure demand and service side effects.
5. Event departure wave	M + C / M	Schedule a venue event; compare stop queues and road clearance.
6. Junction redesign	N + M / M	Change control or lane connections; measure throughput and waiting. [10][20]
7. Roadworks / closure	C / M	Apply a verified access restriction; measure detours and isolation.
8. Traffic collision	N + C / H	Observe native collisions; deterministic spawning needs a verified event adapter. [4]
9. Transit service cut	N + C / M	Reduce line vehicles; compare waiting and mode substitution.
10. Freight bottleneck	N + C / M	Constrain a cargo corridor; observe delivery delays and queues.
11. Parking pressure	N + C / M	Vary available parking; observe destination access and circulation.
12. Emergency corridor	M + C / H	Test junction/access interventions; record response-time distribution.

First choice: scenario 6 has an inspected bridge action and a visible, measurable effect. Scenario 7 is attractive but needs a genuine reversible closure implementation; demolition is not an equivalent temporary restriction.

Scenario ideas: hazards

Native disaster behavior is documented for forest fire, hail, and tornado. The remaining ideas combine existing mechanisms with proposed custom or external models. This list does not claim every hazard has a built-in trigger. [5]

Scenario	Route / effort	Proposed trigger and useful outcome
13. Building fire	N + C / M-H	Verify an incident trigger; measure detection, arrival, control and damage.
14. Forest-edge fire	N + C / M-H	Place a controlled ignition near a district; compare response coverage.
15. Hail disruption	N + C / M-H	Use a verified event route; observe road incidents and repairs.
16. Tornado evacuation	N + C / H	Script a tested event; compare shelter access and evacuation completion.
17. River overflow	M + C / M-H	Increase a water source; map inundation and service loss. [14]
18. Coastal surge proxy	M + C / H	Raise boundary water; compare exposed roads and facilities.
19. Dam-release proxy	M + C / H	Control upstream inflow; measure access loss; no structural failure claim.
20. Urban flash flood	X + C / H	Use external rainfall/runoff and drainage logic; project impacts into the city.
21. Drought / low water	M + C / M	Reduce available water; observe supply deficits and service effects.
22. Heatwave stress	N + C + X / H	Vary temperature-related demand; externally model health exposure.
23. Snow / cold spell	N + C / M	Test road maintenance and energy stress on a suitable map.
24. Earthquake / landslide	X + C / H	Apply an external damage footprint; simulate access and service consequences.

Flood distinction: visible water motion is useful for demonstrating impacts. Rainfall-to-runoff, drainage capacity, flood depth accuracy, and structural damage require separate evidence. A visual tsunami-like wave is not a validated tsunami model.

Scenario ideas: service failures

These designs use city services and infrastructure as dependencies. The native game supplies service, utility, and economic mechanisms; the proposed event sequences and response policies belong to your scenario runner. [6][7][8]

Scenario	Route / effort	Proposed trigger and useful outcome
25. Local blackout	N + C / M	Disable a selected supply path; measure buildings without power and restoration.
26. Transformer bottleneck	N + C / M	Constrain transfer capacity; compare alternative network connections.
27. Water outage	N + C / M	Interrupt a supply source or path; measure people and services affected.
28. Sewage overload	N + C / M	Reduce processing capacity; track backlog and affected districts.
29. Pollution exposure	N + C + X / H	Use pollution state as input; external model estimates exposure, not game health scores.
30. Hospital surge	N + C / H	Inject a synthetic demand wave; compare access and occupancy proxies.
31. Garbage backlog	N + C / M	Reduce collection capacity; measure accumulation and recovery time.
32. Supply interruption	N + C / M	Constrain freight access; monitor shortages and business operation.
33. Crime hotspot	N + C / M-H	Tune a transparent synthetic demand model; measure service access.
34. Disease outbreak	X + C / H	Run a contact or compartment model externally; map mobility and care demand.
35. Evacuation inequity	X + C / H	Assign mobility/compliance cohorts; compare accessibility and unmet demand.
36. Sensor / network outage	C / M	Drop, delay or corrupt synthetic observations; test City Brain's fallback decisions.

Combine causes: flooded road -> delayed ambulance -> hospital access loss -> alternate route. Define the model responsible for each transition and measure it; the whole chain is not automatically implemented by the game.

Sources

Reviewed 5 October 2026. Official diaries describe intended game behavior. Mod descriptions are author claims; code entries identify what was inspected. No game or mod combination was executed. Bridge source links are pinned to the reviewed revision.

[1] Original requested link: Modding API (CS I)

https://skylines.paradoxwikis.com/Modding_API
Access blocked; included to document the edition mismatch, not as verified CS II evidence.

[2] Paradox: Code Modding developer diary

https://www.paradoxinteractive.com/games/cities-skylines-ii/modding/dev-diary-3-code-modding
21 March 2024. Official toolchain, systems, Burst and managed-code patching limits.

[3] Paradox: Citizen Simulation & Lifepath

https://www.paradoxinteractive.com/games/cities-skylines-ii/features/citizen-simulation-lifepath
Native citizen activities and relationships; design documentation, not independent validation.

[4] Paradox: Traffic AI

https://www.paradoxinteractive.com/games/cities-skylines-ii/features/traffic-ai
Native traffic, routing and collision context.

[5] Paradox: Climate & Seasons

https://www.paradoxinteractive.com/games/cities-skylines-ii/features/climate-seasons
Weather, time, documented disaster types and shelter behavior.

[6] Paradox: City Services

https://www.paradoxinteractive.com/games/cities-skylines-ii/features/city-services-districts-policies
Fire response and other native service mechanisms; economic details have later revisions.

[7] Paradox: Electricity & Water

https://www.paradoxinteractive.com/games/cities-skylines-ii/features/electricity-water
Utility networks, service effects and infrastructure context.

[8] Paradox: Economy 2.0, Part 1

https://www.paradoxinteractive.com/games/cities-skylines-ii/news/dev-diary-economy-part-one
3 June 2024. Revisions including fees/toggle for imported services; demonstrates version sensitivity.

[9] Paradox: Adding Custom Assets

https://www.paradoxinteractive.com/games/cities-skylines-ii/news/adding-custom-assets
4 December 2025. Official editor workflow for buildings and props.

Sources / continued

[10] krzychu124: Traffic

https://github.com/krzychu124/Traffic
Author repository; lane connectors, priorities, save behavior and network-edit limitations.

[11] slyh: Traffic Lights Enhancement guide

https://github.com/slyh/Cities2-TrafficLightsEnhancement/blob/master/GUIDE.md
Sequence controls and stated absence of duration control. Repository README cites 1.3.3f1; verify compatibility.

[12] ruzbeh0: Realistic Trips / Time2Work

https://github.com/ruzbeh0/Time2Work
README lists v2.9.3 targeting game 1.6.*; schedules, shifts, school and travel-demand controls.

[13] ruzbeh0: Realistic Workplaces And Households

https://github.com/ruzbeh0/RealisticWorkplacesAndHouseholds
README lists v2.3.5 targeting game 1.6.*; capacity calculations and stabilization caveats.

[14] yenyang: Water Features

https://github.com/yenyang/Water_Features
Water-source tools, seasonal streams, waves/tides, and author-documented limits.

[15] taipei-native: Carto

https://github.com/taipei-native/Carto
Geospatial export formats, projection controls, and distribution instructions.

[16] Carto: API guide

https://github.com/taipei-native/Carto/wiki/Api
Author API documentation for programmatic exports.

[17] LancerComet: CS2MCP

https://github.com/LancerComet/cities-skylines-2-mcp
Unofficial bridge README, tools, build requirements and limitations; repository inspected, not executed.

[18] CS2MCP: bridge lifecycle and queue source

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/BridgeSystem.cs
Read alongside Mod.cs in the same directory; update phase, pause handling and request queue inspected.

Sources / continued

[19] CS2MCP: traffic data source

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/RequestHandlers.Traffic.cs
Inspected road flow/volume and bottleneck queries; game diagnostics, not physical calibration.

[20] CS2MCP: junction control source

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/RequestHandlers.Junctions.cs
Inspected node-upgrade controls; not evidence of arbitrary signal duration control.

[21] LinnIbrahim: MapToSkylines2

https://github.com/LinnIbrahim/CitiesSkylinesOSM
README states map-building mod is in progress despite broader import claims; experimental candidate.

[22] Cities2Modding: MapImageLayer

https://github.com/Cities2Modding/MapImageLayer
Archived 15 April 2024; historical overlay reference.

[23] Eclipse SUMO: TraCI documentation

https://sumo.dlr.de/docs/TraCI/
Primary documentation for external traffic simulation access and control.

[24] US EPA: Storm Water Management Model

https://www.epa.gov/water-research/storm-water-management-model-swmm
Primary overview of runoff and drainage modeling; potential external component.

[25] Paradox: Expansion Pass 2 announcement

https://www.paradoxinteractive.com/media/press-releases/paradox-interactive/cities-skylines-ii-expands-with-parks-tourist-attractions-and-city-services-in-new-expansion-pass
15 September 2026. First Response announced as upcoming content; not assumed available.

[26] JadHajjar: Skyve

https://github.com/JadHajjar/Skyve
Playset management and author-described compatibility catalog.

[27] CS2MCP: build project

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/CS2MCP.Bridge.csproj
Inspected net472 target and game-assembly references; SDK and runtime target are distinct.

Sources / continued

[28] CS I API reference copy

https://wiki.biligame.com/csl/Modding_API
Used to locate the legacy API sections while the original wiki was inaccessible. Examples were cross-checked against author source code below.

[29] TM:PE: simulation callbacks

https://github.com/CitiesSkylinesMods/TMPE/blob/master/TLM/TLM/Lifecycle/ThreadingExtension.cs
Primary code: OnBeforeSimulationTick, OnBeforeSimulationFrame, and OnAfterSimulationTick.

[30] TM:PE: save-data extension

https://github.com/CitiesSkylinesMods/TMPE/blob/master/TLM/TLM/Lifecycle/SerializableDataExtension.cs
Primary code: OnLoadData, OnSaveData, LoadData and SaveData.

[31] boformer: CS I mod lifecycle example

https://gist.github.com/boformer/3e01770a0c86cf9273ccdcc33d9bbc42
Author code demonstrating IUserMod, LoadingExtensionBase and OnLevelLoaded.

[32] CS2MCP: route dispatch, state and pause controls

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/RequestHandlers.cs
Inspected routes, parameter names, state fields and errors used in the PowerShell and Python examples.

[33] CS2MCP: timed runs and saving

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/RequestHandlers.Meta.cs
Inspected frame conversion, auto-pause target, cancel operation and asynchronous save behavior.

[34] CS2MCP: entity queries and building data

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/RequestHandlers.Build.cs
Primary code supporting the building-query pattern and entity identity discussion.

[35] CS2MCP: HTTP listener

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/HttpBridgeServer.cs
Inspected loopback listener, liveness path and request parsing.

[36] CS2MCP: mod registration

https://github.com/LancerComet/cities-skylines-2-mcp/blob/f0894c1831153742d2a586b830418177839a2428/CS2MCP.Bridge/Mod.cs
Primary code demonstrating IMod.OnLoad, UpdateAt and OnDispose.